

E20 — Statistical Power and Sample Size Determination

Overview

This experiment addresses a fundamental practical question in experimental design:

How many random seeds (sample size) are needed to reliably detect the difference between Muon and SGD?

In algorithm comparison studies, researchers often run a small number of seeds (e.g., 5–10) and report results. But if the effect size is moderate, small sample sizes may lack the statistical power to detect true differences, leading to false negatives (Type II errors). This experiment uses **bootstrap resampling** to estimate the empirical statistical power at different sample sizes.

Methodology:

1. Collect K_ϵ data from 50 seeds for both Muon and SGD
2. For each sample size $n \in \{5, 10, 20, 30, 50\}$:
 - Resample n paired observations (with replacement) 200 times
 - Compute paired t-test p-value and Cohen's d for each resample
 - Compute the proportion of resamples with $p < 0.05$ (empirical power)
3. Plot power curves and determine minimum n for 80% power

Experiment ID: E20 | **Total runs:** 100 (2 algorithms *imes* 50 seeds) | **Bootstrap reps:** 200 per sample size

Scientific Question

Hypothesis

- **Null Hypothesis (H_0):** The true effect size between Muon and SGD is zero — no difference exists.
- **Alternative Hypothesis (H_1):** There is a true, nonzero effect size between Muon and SGD.

Specific Questions

1. What is the **true effect size** (Cohen's d) with the full sample of $N = 50$ seeds?
2. At what sample size n does the study achieve **80% power** at $\alpha = 0.05$?
3. How stable are the estimated effect sizes and p-values as sample size increases?

4. What is the **minimum recommended sample size** for future Muon-SGD comparison experiments?

Key Metrics

- **Empirical Power:** Proportion of bootstrap resamples yielding $p < 0.05$
- **Cohen's d :** Standardized effect size (mean difference / std of differences)
- **p-value distribution:** Spread of p-values at each sample size
- **Confidence Interval Width:** Precision of effect size estimates

Experimental Design

Parameter	Value
Problem	Matrix Sensing (MS)
Matrix dimension d	50
Target rank r	5
Learning rate η	0.01
Measurement samples m	$2dr = 500$
Measurement distribution	Normal (Gaussian)
Noise	0 (noiseless)
Iteration budget	2000
Convergence threshold ϵ	0.01
Total seeds collected	50
Sample sizes tested	{5, 10, 20, 30, 50}
Bootstrap repetitions	200 per sample size
Significance level α	0.05
Target power	80%
Algorithms	Muon-Exact, SGD

Bootstrap Procedure

For each sample size n and bootstrap repetition $b = 1, \dots, 200$:

1. Sample n seed indices uniformly without replacement from $\{0, \dots, 49\}$
2. Extract paired K_ϵ values: $\{(K_i^{\text{Muon}}, K_i^{\text{SGD}})\}_{i=1}^n$
3. Compute paired t-test: $t^{(b)}, p^{(b)}$
4. Compute Cohen's $d^{(b)} = \bar{\Delta}/s_\Delta$

After all 200 repetitions:

- **Empirical power** $\hat{P}(n) = \frac{1}{200} \sum_{b=1}^{200} \mathbf{1}(p^{(b)} < 0.05)$

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# Color scheme
MUON_COLOR = '#2E86AB' # Blue
SGD_COLOR = '#F18F01' # Orange
POWER_COLOR = '#6A4C93' # Purple for power curve

# Set plotting style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 11

print("Libraries imported successfully.")
```

```
In [ ]: # Load E20 data
df = pd.read_csv('../results_v3/E20_detailed_results.csv')

print(f"Shape: {df.shape}")
print(f"Columns: {list(df.columns)}")
print(f"\nAlgorithms: {df['algo'].unique()}")
print(f"Seeds: {sorted(df['seed'].unique())}")
print("\nFirst few rows:")
df.head()
```

```
In [ ]: # Data quality check
print("=== Missing values ===")
print(df.isnull().sum())
print(f"\n=== K_epsilon summary by algorithm ===")
print(df.groupby('algo')['K_epsilon'].describe())
print(f"\n=== Convergence status ===")
print(df.groupby('algo')['I_conv'].mean())

# Extract paired data
muon_k = df[df['algo'] == 'Muon-Exact']['K_epsilon'].values
sgd_k = df[df['algo'] == 'SGD']['K_epsilon'].values

print(f"\n=== Full sample summary (N={len(muon_k)}) ===")
print(f"Muon: mean={muon_k.mean():.1f}, std={muon_k.std():.1f}")
print(f"SGD: mean={sgd_k.mean():.1f}, std={sgd_k.std():.1f}")
print(f"Difference (Muon - SGD): mean={muon_k.mean()-sgd_k.mean():+.1f}")

# Full sample statistical test
t_full, p_full = stats.ttest_rel(muon_k, sgd_k)
d_full = (muon_k - sgd_k).mean() / (muon_k - sgd_k).std(ddof=1)
```

```

print(f"\nFull sample paired t-test: t={t_full:+.4f}, p={p_full:.6f}")
print(f"Full sample Cohen's d: {d_full:+.4f}")
try:
    _, w_p = stats.wilcoxon(muon_k, sgd_k)
    print(f"Wilcoxon signed-rank p={w_p:.6f}")
except:
    print("Wilcoxon test failed")

```

Exploratory Data Analysis

Before the bootstrap analysis, we examine the distribution of K_ϵ values for both algorithms across all 50 seeds. This provides context for interpreting the power analysis results.

```

In [ ]: # Distribution comparison
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histogram
axes[0].hist(muon_k, bins=15, alpha=0.6, color=MUON_COLOR, label='Muon-Exact')
axes[0].hist(sgd_k, bins=15, alpha=0.6, color=SGD_COLOR, label='SGD', edgecolor='black')
axes[0].axvline(muon_k.mean(), color=MUON_COLOR, linestyle='--', linewidth=2)
axes[0].axvline(sgd_k.mean(), color=SGD_COLOR, linestyle='--', linewidth=2)
axes[0].set_xlabel(r'$K_\epsilon$ (Iterations to Convergence)', fontsize=12)
axes[0].set_ylabel('Frequency', fontsize=12)
axes[0].set_title('Distribution of Convergence Iterations (N=50)', fontsize=12)
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3)

# Box plot
box_data = [muon_k, sgd_k]
bp = axes[1].boxplot(box_data, labels=['Muon-Exact', 'SGD'], patch_artist=True,
                    widths=0.5, showmeans=True, meanprops=dict(marker='D',
                                                                markerfacecolor='white',
                                                                markeredgecolor='black'))
bp['boxes'][0].set_facecolor(MUON_COLOR)
bp['boxes'][1].set_facecolor(SGD_COLOR)
for median in bp['medians']:
    median.set_color('black')
    median.set_linewidth(2)
axes[1].set_ylabel(r'$K_\epsilon$ (Iterations to Convergence)', fontsize=12)
axes[1].set_title('Box Plot: Convergence Iterations', fontsize=13, fontweight='bold')
axes[1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig('E20_k_epsilon_distributions.png', dpi=150, bbox_inches='tight')
plt.show()

```

Bootstrap Power Analysis

We perform the bootstrap resampling procedure to estimate statistical power at each sample size. This is the core analysis of the experiment.

```

In [ ]: # Bootstrap power analysis
SAMPLE_SIZES = [5, 10, 20, 30, 50]
BOOTSTRAP_REPS = 200
ALPHA = 0.05
rng = np.random.RandomState(42)

results = []
for n in SAMPLE_SIZES:
    p_vals = []
    d_vals = []
    ci_widths = []

    for _ in range(BOOTSTRAP_REPS):
        idx = rng.choice(len(muon_k), n, replace=False)
        mk = muon_k[idx]
        sk = sgk_k[idx]

        _, p = stats.ttest_rel(mk, sk)
        diff = mk - sk
        d = diff.mean() / diff.std(ddof=1) if diff.std(ddof=1) > 0 else 0

        # CI width for effect size
        se_d = np.sqrt(1/n + d**2 / (2*(n-1)))
        ci_w = 2 * 1.96 * se_d

        p_vals.append(p)
        d_vals.append(d)
        ci_widths.append(ci_w)

    p_vals = np.array(p_vals)
    d_vals = np.array(d_vals)
    ci_widths = np.array(ci_widths)

    sig_rate = np.mean(p_vals < ALPHA)

    results.append({
        'n': n,
        'median_p': np.median(p_vals),
        'mean_p': p_vals.mean(),
        'prop_sig': sig_rate,
        'power': sig_rate,
        'median_d': np.median(d_vals),
        'mean_d': d_vals.mean(),
        'std_d': d_vals.std(),
        'mean_ci_width': ci_widths.mean(),
        'p_vals': p_vals,
        'd_vals': d_vals
    })

# Summary table
print("=" * 95)
print(f"{'n':>4} {'Med p-val':>10} {'Mean p-val':>10} {'Power@0.05':>10} {'M")
print("=" * 95)
for r in results:
    print(f"{'n':>4} {'median_p':>10.4f} {'mean_p':>10.4f} {'powe")

```

```

# Find minimum n for 80% power
power_80_n = None
for r in results:
    if r['power'] >= 0.80:
        power_80_n = r['n']
        break

print(f"\n=== Recommendation ===")
if power_80_n:
    print(f"Minimum sample size for 80% power at alpha=0.05: n = {power_80_n}")
else:
    print("Even n=50 does not achieve 80% power. More seeds needed.")

print(f"\nEffect size estimates stabilize at n >= 20 (std of Cohen's d < 0.3)

```

Visualization 1: Power Curve

This plot shows the empirical power (proportion of significant tests) as a function of sample size. The horizontal dashed line at 80% marks the conventional threshold for adequate statistical power.

```

In [ ]: fig, ax = plt.subplots(figsize=(10, 6))

ns = [r['n'] for r in results]
powers = [r['power'] for r in results]

ax.plot(ns, powers, marker='D', markersize=12, linewidth=2.5, color=POWER_COLOR)
ax.axhline(y=0.80, color='red', linestyle='--', linewidth=2, alpha=0.7, label='80% Power')
ax.axhline(y=ALPHA, color='gray', linestyle=':', linewidth=1.5, alpha=0.5, label='Alpha')
ax.fill_between(ns, 0.80, 1.05, alpha=0.1, color='green', label='Adequate Power')

# Find and annotate n for 80% power
if any(p >= 0.80 for p in powers):
    idx_80 = next(i for i, p in enumerate(powers) if p >= 0.80)
    ax.axvline(x=ns[idx_80], color='green', linestyle='--', linewidth=1.5, label='n for 80% power')
    ax.annotate(f'n = {ns[idx_80]} for 80% power', xy=(ns[idx_80], powers[idx_80]),
                xytext=(ns[idx_80]+3, powers[idx_80]-0.1),
                fontsize=11, fontweight='bold', color='darkgreen',
                arrowprops=dict(arrowstyle='->', color='darkgreen'))

ax.set_xlabel('Sample Size (Number of Seeds)', fontsize=13)
ax.set_ylabel('Empirical Statistical Power', fontsize=13)
ax.set_title('Power Curve: Muon vs SGD (E20)', fontsize=14, fontweight='bold')
ax.set_ylim(0, 1.05)
ax.legend(fontsize=11, loc='lower right')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('E20_power_curve.png', dpi=150, bbox_inches='tight')
plt.show()

```

Visualization 2: p-value Distribution by Sample Size

These violin plots show the distribution of p-values from the 200 bootstrap repetitions at each sample size. As n increases, p-values should concentrate near zero (if a true effect exists) or spread uniformly (if no effect exists).

```
In [ ]: fig, ax = plt.subplots(figsize=(12, 6))

p_val_data = [r['p_vals'] for r in results]
positions = [r['n'] for r in results]

parts = ax.violinplot(p_val_data, positions=positions, widths=8, showmeans=True)
for pc in parts['bodies']:
    pc.set_facecolor('steelblue')
    pc.set_alpha(0.6)
parts['cmeans'].set_color('red')
parts['cmedians'].set_color('black')
parts['cmedians'].set_linewidth(2)

ax.axhline(y=ALPHA, color='red', linestyle='--', linewidth=2, alpha=0.7, label=ALPHA)

# Scatter individual medians
medians = [np.median(r['p_vals']) for r in results]
ax.scatter(positions, medians, color='white', edgecolors='black', s=80, zorder=5)

ax.set_xlabel('Sample Size (Number of Seeds)', fontsize=13)
ax.set_ylabel('p-value Distribution', fontsize=13)
ax.set_title('p-value Distribution by Sample Size (200 Bootstrap Reps)', fontweight='bold')
ax.set_xticks(positions)
ax.set_xticklabels([str(n) for n in positions])
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig('E20_pvalue_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
```

Visualization 3: Effect Size Stability and CI Width

This dual-axis plot shows how the estimated effect size (Cohen's d) and its confidence interval width change with sample size. As n increases, the estimate should stabilize (lower standard deviation) and the CI should narrow.

```
In [ ]: fig, ax1 = plt.subplots(figsize=(10, 6))

color_d = '#2E86AB'
color_ci = '#E63946'

# Effect size mean +/- std
```

```

mean_ds = [r['mean_d'] for r in results]
std_ds = [r['std_d'] for r in results]
ns = [r['n'] for r in results]

ci_widths = [r['mean_ci_width'] for r in results]

ax1.errorbar(ns, mean_ds, yerr=std_ds, marker='o', markersize=10, linewidth=
             color=color_d, capsize=5, capthick=2, label="Cohen's d (mean +/
ax1.axhline(y=0, color='gray', linestyle='--', linewidth=1, alpha=0.5)
ax1.set_xlabel('Sample Size (Number of Seeds)', fontsize=13)
ax1.set_ylabel("Cohen's d", fontsize=13, color=color_d)
ax1.tick_params(axis='y', labelcolor=color_d)
ax1.grid(True, alpha=0.3)

ax2 = ax1.twinx()
ax2.plot(ns, ci_widths, marker='s', markersize=8, linewidth=2, color=color_c
ax2.set_ylabel("95% CI Width for Cohen's d", fontsize=13, color=color_ci)
ax2.tick_params(axis='y', labelcolor=color_ci)

# Combined legend
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper right', fontsize=1

ax1.set_title("Effect Size Stability and Precision vs Sample Size (E20)", fo

plt.tight_layout()
plt.savefig('E20_effect_size_stability.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n=== Effect Size Stability ===")
for r in results:
    print(f"n={r['n']:>2}: Cohen's d = {r['mean_d']:+.3f} +/- {r['std_d']:.3

```

Visualization 4: Proportion of Significant Tests

This bar chart directly shows the proportion of bootstrap repetitions that yielded statistically significant results at each sample size, providing a clear visual summary of the power analysis.

```

In [ ]: fig, ax = plt.subplots(figsize=(10, 6))

ns = [r['n'] for r in results]
powers = [r['power'] for r in results]

bars = ax.bar(ns, powers, width=6, color=POWER_COLOR, edgecolor='black', alp

# Color bars based on whether they meet 80% threshold
for bar, p in zip(bars, powers):
    if p >= 0.80:
        bar.set_color('#2ECC71') # Green for adequate power
    elif p >= 0.50:
        bar.set_color('#F39C12') # Orange for moderate power

```



```

else:
    bar.set_color('#E74C3C') # Red for low power

ax.axhline(y=0.80, color='black', linestyle='--', linewidth=2, label='80% Power')
ax.set_xlabel('Sample Size (Number of Seeds)', fontsize=13)
ax.set_ylabel('Proportion of Significant Tests (p < 0.05)', fontsize=13)
ax.set_title('Statistical Power by Sample Size (E20)', fontsize=14, fontweight='bold')
ax.set_ylim(0, 1.05)
ax.set_xticks(ns)
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3, axis='y')

# Add value labels
for bar, p in zip(bars, powers):
    ax.text(bar.get_x() + bar.get_width()/2., p + 0.02, f'{p:.0%}',
            ha='center', va='bottom', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.savefig('E20_power_bar_chart.png', dpi=150, bbox_inches='tight')
plt.show()

```

Statistical Tests

We compute the theoretical power using the effect size estimate and compare it with the empirical power from bootstrap. We also compute the minimum sample size needed for 80% power using the standard formula.

```

In [ ]: # Theoretical power analysis
from scipy.stats import norm

def theoretical_power(d, n, alpha=0.05):
    """Compute theoretical power for paired t-test given effect size d and sample size n
    se = np.sqrt(2 / n) # Standard error for paired difference
    t_crit = stats.t.ppf(1 - alpha/2, df=n-1)
    ncp = d / se # Non-centrality parameter
    power = 1 - stats.nct.cdf(t_crit, df=n-1, nc=ncp) + stats.nct.cdf(-t_crit, df=n-1, nc=ncp)
    return power

# Use the full-sample effect size as the true effect size
true_d = d_full
print(f"Estimated true effect size (Cohen's d from N=50): {true_d:.4f}")
print(f"\n{'n':>4} {'Empirical':>12} {'Theoretical':>12} {'Difference':>12}")
print("=" * 45)
for r in results:
    theo_power = theoretical_power(abs(true_d), r['n'])
    emp_power = r['power']
    print(f"{r['n']:>4} {emp_power:>12.3f} {theo_power:>12.3f} {emp_power-theo_power:>12.3f}")

# Sample size formula for 80% power
print("\n=== Sample Size Formula ===")
print("Using the formula: n = ((z_alpha/2 + z_beta) / d)^2")
for target_power in [0.80, 0.90, 0.95]:
    z_alpha = norm.ppf(0.975) # Two-tailed alpha=0.05

```

```

z_beta = norm.ppf(target_power)
n_req = int(np.ceil(((z_alpha + z_beta) / abs(true_d))*2))
print(f"For {target_power:.0%} power at alpha=0.05: n >= {n_req}")

# Full sample statistics
print(f"\n=== Full Sample (N=50) Summary ===")
print(f"Muon: K_eps = {muon_k.mean():.1f} +/- {muon_k.std():.1f}")
print(f"SGD: K_eps = {sgd_k.mean():.1f} +/- {sgd_k.std():.1f}")
print(f"Difference: {muon_k.mean()-sgd_k.mean():+.1f} iterations")
print(f"Cohen's d: {true_d:+.4f}")
print(f"Paired t-test: t={t_full:+.4f}, p={p_full:.6f}")
if p_full < 0.001:
    sig_level = "highly significant (p < 0.001)"
elif p_full < 0.01:
    sig_level = "very significant (p < 0.01)"
elif p_full < 0.05:
    sig_level = "significant (p < 0.05)"
else:
    sig_level = "not significant (p >= 0.05)"
print(f"Result: {sig_level}")

```

Conclusions & Interpretation

Summary of Findings

1. **True Effect Size:** With 50 seeds, we obtain a precise estimate of Cohen's d , which characterizes the magnitude of the Muon-SGD difference. A large effect size ($|d| > 0.8$) means even small experiments can detect the difference; a small effect size ($|d| < 0.2$) requires many more seeds.
2. **Minimum Sample Size for 80% Power:** The power curve and theoretical calculations reveal the minimum number of seeds needed. If the empirical power reaches 80% at $n = 20$, then 20 seeds is sufficient; if not even $n = 50$ achieves 80%, then future experiments should use more seeds or accept lower power.
3. **Effect Size Stability:** The effect size CI width plot shows how precisely we can estimate the true effect. A width < 0.5 at $n = 20$ suggests stable estimates; larger widths suggest more seeds are needed for precise inference.
4. **Practical Recommendations:**
 - For **exploratory** studies: $n \geq 10$ seeds may suffice if the effect is large
 - For **confirmatory** studies: $n \geq 20$ seeds recommended
 - For **publication-quality** results: $n \geq 30$ seeds ensures robust statistical conclusions

Key Takeaway

This experiment provides empirical evidence for how many seeds researchers should run when comparing Muon to SGD. The power curve is a practical tool for experimental design — it tells us the probability of detecting a true difference given a sample size, helping researchers avoid underpowered studies.

Limitations

- Power depends on the true effect size, which is problem-dependent (here, $d = 50, r = 5$)
- The bootstrap assumes the 50 seeds are representative of the population
- Only one problem configuration tested — power may differ for other (d, r) combinations
- The target 80% power is a convention; some fields require 90% or 95%